

LUA

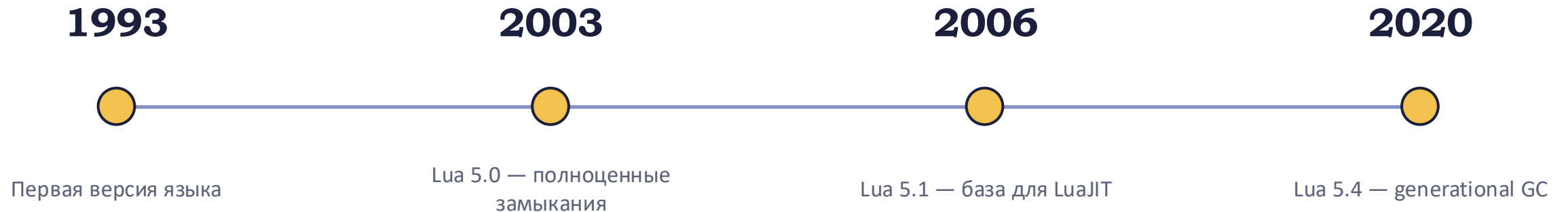
История и особенности · Запуск программы · Типы и переменные ·
Функции, рекурсия и замыкания

Команда 6: Кулиев Хакберди, Кретьова Дарья, Эйсмонт Александра



История

- Создан в 1993 году в PUC-Rio (Католический университет Рио-де-Жанейро, Бразилия)
- Авторы: Roberto Ierusalimschy, Luiz Henrique de Figueiredo, Waldemar Celes
- Изначально — встраиваемый DSL для конфигурации промышленных систем (проекты Petrobras)
- Название означает «луна» (порт. Lua)





Особенности и спецификация



Компактность

Интерпретатор — сотни КБ, легко встраивается в C/C++



Table

Единая структура данных: массив + словарь одновременно



Прототипное ООП

Классов нет «из коробки» — используются метатаблицы



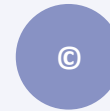
Корутины

Встроенная кооперативная многозадачность вместо потоков ОС



Сборка мусора

Инкрементальный / generational (с 5.4) GC



Lua 5.4 · MIT

Единственная референсная реализация на ANSI C



Способы запуска — сравнение

Три варианта для команды — выбор по скорости старта, воспроизводимости версии и удобству демонстрации

Способ	Плюсы	Минусы
Docker	Одинаковая версия у всех участников, изоляция от локальной системы	Нужен установленный Docker, чуть медленнее старт контейнера
Онлайн-среда	Ничего не нужно устанавливать, быстро для демонстрации на защите	Ограничения по файловому вводу-выводу и подключению модулей
Локальное окружение	Полный контроль, IDE, менеджер пакетов LuaRocks	Требуется установка и настройка окружения вручную



Запуск через Docker

- Официального образа Lua в Docker Hub нет — используют образы сообщества
- Популярный вариант: nickblah/lua (регулярно обновляется)
- Плюс: у всех участников команды одна и та же версия Lua

Быстрый запуск

```
docker run -it --rm \
  nickblah/lua:5.4 lua
```

Dockerfile проекта

```
FROM nickblah/lua:5.4
WORKDIR /app
COPY . .
CMD ["lua", "main.lua"]

-- сборка и запуск:
docker build -t lua-app .
docker run --rm lua-app
```



Онлайн-среда и локальное окружение



Онлайн

- lua.org/demo.html — официальная демо-страница, запуск в браузере
- replit.com — песочница без установки
- onecompiler.com/lua
- tio.run



Локально

- `apt install lua5.4` (Linux) / `brew install lua` (macOS) / сборка из исходников (Windows)
- Пакетный менеджер LuaRocks: `luarocks install <pkg>`
- Проверка версии: `lua -v`, запуск: `lua script.lua`
- Редакторы: VS Code (расширение Lua), ZeroBrane Studio



Типы данных

- 8 базовых типов, динамическая типизация — тип хранится у значения, не у переменной
- Проверка типа: функция `type(x)`

`nil`

отсутствие значения

`boolean`

true / false

`number`

целые и вещественные

`string`

неизменяемые строки

`table`

массив + словарь

`function`

функции первого класса

`userdata`

данные из C

`thread`

корутина



Операции над типами

`+` `-` `*` `/` `//` `%` `^`

Арифметические операции

`..`

Конкатенация строк

`==` `~=` `<` `>` `<=` `>=`

Операции сравнения

`and` `/` `or` `/` `not`

Логические — работают с любыми типами, не только `boolean`



Приведение типов

- Явное преобразование: `tonumber("42")`, `tostring(42)`
- Неявное приведение в арифметике: `"10" + 5 == 15`
- Единственные «ложные» значения в условиях — `nil` и `false`
- `0` и `""` — истина (в отличие от многих других языков)

Примеры

```
print(tonumber("42") + 1)    --> 43
print("10" + 5)              --> 15
print("a" .. 1)              --> a1

if 0 then print("истина") end
--> истина (0 не является ложным)
```



Области видимости

- По умолчанию переменные глобальные — нужно явно писать `local`
- Лексическая (статическая) область видимости
- Блоки задаются конструкцией `do ... end`

Пример

```
local x = 10
do
  local x = 20  -- новая переменная,
                -- видна только внутри блока
end
print(x)  --> 10
```



Владение и передача владения

- number / string / boolean / nil — передаются по значению (копия)
- table / function — передаются по ссылке
- Понятия «владения» как в Rust нет — за освобождение памяти отвечает сборщик мусора

Пример

```
local function change(t)
    t.x = 100
end

local a = {x = 1}
change(a)
print(a.x)    --> 100
```



Функции: объявление и вызов

- Функции — значения первого класса (тип function)
- Можно объявлять именованные и анонимные функции
- Хранятся в переменных, передаются как аргументы

Пример

```
local function greet(name)
  return "Hello, " .. name
end

local f = function(x)
  return x * 2
end -- анонимная функция
```



Входные данные функций

- Позиционные параметры без строгой типизации
- Значение по умолчанию эмулируется через or: `local x = x or 10`
- Переменное число аргументов через ...

Пример

```
local function sum(...)
  local s = 0
  for _, v in ipairs({...}) do
    s = s + v
  end
  return s
end
print(sum(1, 2, 3)) --> 6
```



Выходные данные функций

- Функция может вернуть несколько значений одновременно
- Лишние возвращаемые значения отбрасываются
- Недостающие значения становятся nil

Пример

```
local function minmax(t)
    return math.min(table.unpack(t)),
           math.max(table.unpack(t))
end

local lo, hi = minmax({3, 1, 4, 1, 5})
print(lo, hi) --> 1 5
```

Рекурсия

- Хвостовой вызов — когда рекурсивный вызов является последним действием функции; такие вызовы Lua оптимизирует и не растит стек
- Для рекурсии внутри себя — `local function`, а не `local f = function`

Обычная рекурсия (не хвостовая)

```
local function fact(n)
    if n == 0 then return 1 end
    return n * fact(n - 1)
end
print(fact(5)) --> 120
```

Хвостовая рекурсия (с аккумулятором)

```
local function factTail(n, acc)
    acc = acc or 1
    if n == 0 then return acc end
    return factTail(n - 1, n * acc)
end
print(factTail(5)) --> 120
```



Замыкания

- Функция «запоминает» переменные из внешней области видимости (upvalues)
- Каждый вызов внешней функции создаёт новое, независимое замыкание
- Используются для инкапсуляции состояния без классов

Пример

```
local function counter()
    local n = 0
    return function()
        n = n + 1
        return n
    end
end

local c = counter()
print(c(), c(), c()) --> 1 2 3
```


Спасибо за внимание

